Git Version Control & Repository Management Guide

Basic to Advanced Practical Commands for GitHub, LinkedIn Portfolio, Resume Projects, and DevOps Learning

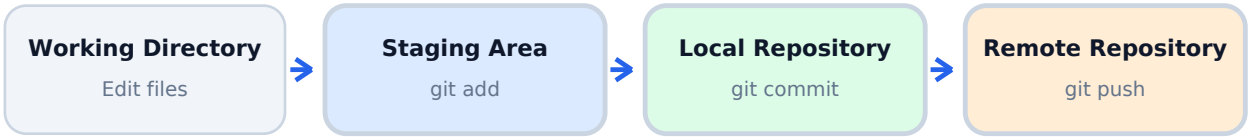
Prepared for: Ravi Shete	Focus: Practical Git CLI Workflow

Table of Contents

---

1. Git Concepts and Snapshot Workflow

Professional Git Snapshot Workflow



Git is a distributed version control system used to track source-code changes, collaborate with teams, and manage project history. In a professional workflow, code moves from the working directory to the staging area, then into the local repository, and finally to the remote repository such as GitHub.

Important concept: A commit is a permanent snapshot of your project at a specific point in time. It helps you recover previous versions, review changes, and collaborate safely.

Core Areas Covered

Command	Purpose / Use Case
`Creating Snapshots`	Initialize repositories, stage files, commit changes, restore files, and manage tracked/untracked files.
`Browsing History`	Review commit logs, inspect changes, compare commits, and identify contributors or line authors.
`Branching & Merging`	Create separate development lines, merge features, stash unfinished work, rebase, and cherry-pick commits.
`Collaboration`	Clone repositories, fetch/pull updates, push changes, manage remotes, tags, and remote branches.
`Rewriting History`	Use reset, revert, amend, reflog, and interactive rebase carefully to correct mistakes.

2. Creating Snapshots: Repository Setup

Initialize a repository when you start a new project or want to start tracking an existing folder with Git.

```
mkdir my-project
cd my-project
git init
```

After running `git init`, Git creates a hidden `.git` folder. This folder stores metadata, commit history, branches, objects, and configuration.

Status Commands

Command	Purpose / Use Case
`git status`	Displays detailed status of working directory and staging area.
`git status -s`	Displays short status, useful for quick review before commit.
`M`	Modified file.
`A`	Added file.
`??`	Untracked file not yet added to Git.

Professional habit: Always run `git status` before `git add`, before `git commit`, and before `git push`.

Staging Files

Command	Purpose / Use Case
`git add file1.js`	Stage a single file.
`git add file1.js file2.js`	Stage multiple specific files.
`git add *.js`	Stage files matching a pattern.
`git add .`	Stage all changes in the current directory.

3. Committing Changes and Managing Files

A commit should represent one meaningful change. Professional commit messages are short, specific, and action-oriented.

```
git commit -m "Add login page"
git commit -m "Fix navbar alignment issue"
git commit -m "Update README with installation steps"
```

Command	Purpose / Use Case
`git commit -m "Message"`	Commit staged files with one-line message.
`git commit`	Open default editor to write a longer commit message.
`git commit -am "Message"`	Stage and commit already tracked modified files. Does not include new files.

File Management Commands

Command	Purpose / Use Case
`git rm file1.js`	Remove file from working directory and Git tracking.
`git rm --cached file1.js`	Remove from Git tracking only, keep file locally. Useful for .env or unwanted tracked files.
`git mv oldname.js newname.js`	Rename or move file and stage the change automatically.

Viewing Differences

Command	Purpose / Use Case
`git diff`	Show unstaged changes.
`git diff --staged`	Show staged changes ready to be committed.
`git diff --cached`	Same purpose as git diff --staged.

4. Undoing Changes Safely

Git provides multiple ways to undo work. Use restore for local file changes, clean for untracked files, reset for commit-level changes, and revert for safe shared-history rollback.

Command	Purpose / Use Case
`git restore --staged file.js`	Unstage a file without deleting local changes.
`git restore file.js`	Discard local changes in one tracked file.
`git restore .`	Discard all tracked local changes.
`git clean -fd`	Remove untracked files and directories. Use carefully.
`git restore --source=HEAD~2 file.js`	Restore file from an older commit.

Safety note: Before using destructive commands like `git clean -fd` or `git reset --hard`, run `git status` and make sure no important uncommitted work will be deleted.

5. Browsing Commit History

Command	Purpose / Use Case
`git log`	Show full commit history.
`git log --oneline`	Show compact one-line commit history.
`git log --reverse`	Show commits from oldest to newest.
`git log --stat`	Show modified files and change statistics.
`git log --patch`	Show actual code changes in commits.

6. Filtering and Inspecting History

Command	Purpose / Use Case
`git log -3`	Show last three commits.
`git log --author="Ravi"`	Filter commits by author.
`git log --before="2026-05-01"`	Show commits before a date.
`git log --after="one week ago"`	Show commits after a relative date.
`git log --grep="login"`	Search commit messages.
`git log -S"login"`	Search code changes containing a specific string.
`git log file.txt`	Show commits that touched a file.
`git blame file.txt`	Show author and commit for each line in a file.

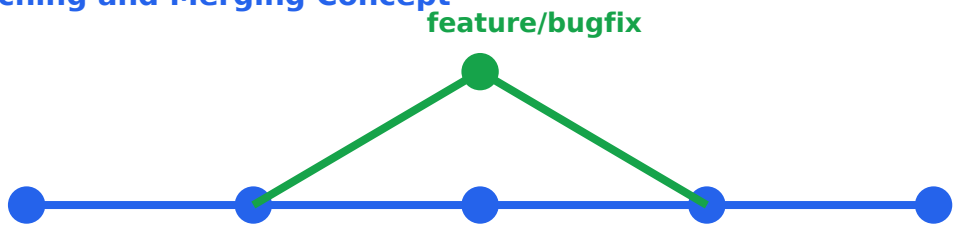
Commit Inspection and Comparison

Command	Purpose / Use Case
`git show HEAD`	Show latest commit details.
`git show HEAD~2`	Show commit two steps before the latest commit.
`git show HEAD:file.js`	Show file version stored in the latest commit.
`git diff HEAD~2 HEAD`	Compare two commits.
`git diff HEAD~2 HEAD file.txt`	Compare one file between two commits.

Advanced debugging: `git bisect` **helps find the commit that introduced a bug by testing good and bad commits step by step.**

7. Branching, Merging and Stashing

Branching and Merging Concept



Create separate branch -> work safely -> merge changes back to main branch

Command	Purpose / Use Case
`git branch bugfix`	Create a new branch.
`git checkout bugfix`	Switch to branch using older syntax.
`git switch bugfix`	Switch to branch using modern syntax.
`git switch -C bugfix`	Create and switch to branch.
`git branch -d bugfix`	Delete a local branch.

Branch naming examples: `feature-login`, `bugfix-navbar`, `dev`, `testing`, `production`.

Stashing Temporary Work

Command	Purpose / Use Case
`git stash push -m "New tax rules"`	Save current unfinished changes temporarily.
`git stash list`	List saved stashes.
`git stash show 1`	Show stash summary.
`git stash apply 1`	Apply stash to working directory.
`git stash drop 1`	Delete a stash.
`git stash clear`	Delete all stashes.

8. Merge, Rebase and Cherry-pick

Command	Purpose / Use Case
`git merge bugfix`	Merge bugfix branch into current branch.
`git merge --no-ff bugfix`	Create a merge commit even when fast-forward is possible.
`git merge --squash bugfix`	Combine all branch commits into one staged change.
`git merge --abort`	Abort merge during conflict.
`git branch --merged`	Show merged branches.
`git branch --no-merged`	Show unmerged branches.
`git rebase main`	Move current branch commits on top of latest main.
`git cherry-pick dad47ed`	Apply one specific commit to current branch.

Merge vs Rebase: Merge preserves branch history. Rebase creates a cleaner linear history. Avoid rebasing shared branches unless you clearly understand the impact.

Typical feature branch workflow

```
git switch main
git pull
git switch -C feature-login
# make changes
git add .
git commit -m "Add login feature"
git push -u origin feature-login
```

9. Collaboration with GitHub and Remote Repositories

Command	Purpose / Use Case
`git clone url`	Download remote repository to local system.
`git fetch origin`	Download latest remote changes without merging.
`git pull`	Fetch and merge remote changes.
`git push origin main`	Upload local commits to remote main branch.
`git push`	Push to configured upstream branch.

Remote and Branch Sharing

Command	Purpose / Use Case
`git remote`	Show remote names.
`git remote -v`	Show remote URLs.
`git remote add upstream url`	Add another remote repository.
`git remote rm upstream`	Remove a remote.
`git branch -r`	Show remote tracking branches.
`git branch -vv`	Show local branch tracking details.
`git push -u origin bugfix`	Push branch and set upstream.
`git push -d origin bugfix`	Delete remote branch.

Tags for Releases

Command	Purpose / Use Case
`git tag v1.0`	Tag latest commit.
`git tag v1.0 commit_id`	Tag an older commit.
`git tag`	List all tags.
`git tag -d v1.0`	Delete local tag.
`git push origin v1.0`	Push tag to remote.
`git push origin --delete v1.0`	Delete remote tag.

10. Rewriting History and Recovery

Command	Purpose / Use Case
`git reset --soft HEAD^`	Remove last commit but keep changes staged.
`git reset --mixed HEAD^`	Remove last commit and unstage changes.
`git reset --hard HEAD^`	Remove last commit and delete changes. Dangerous if work is not backed up.
`git revert commit_id`	Create a new commit that reverses an old commit.
`git revert HEAD~3..`	Revert the last three commits.
`git revert --no-commit HEAD~3..`	Revert changes without creating commits immediately.
`git reflog`	Show history of HEAD movement. Useful for recovery.
`git reflog show bugfix`	Show history of branch pointer.
`git commit --amend`	Edit last commit message or add forgotten files.
`git rebase -i HEAD~5`	Interactively edit, squash, reorder, or delete recent commits.

Professional rule: Use `git revert` for public/shared branches. Use `git reset` mainly for local work before pushing.

11. Best Professional Git Workflow for Project Upload

Use this workflow when publishing your Git project to GitHub for portfolio, LinkedIn, or resume showcase.

```
git init
git status
git add .
git commit -m "Initial commit - Git Version Control Project"
git branch -M main
git remote add origin https://github.com/username/repository.git
git push -u origin main
```

Daily development workflow

```
git pull
git switch -C feature-name
# make changes
git add .
git commit -m "Add feature"
git push -u origin feature-name
```

Recommended Project Title

Git Version Control & Repository Management Project

Command	Purpose / Use Case
Git Version Control Practical Implementation	Good for beginner-to-intermediate project documentation.
Git Repository Management Hands-on Project	Good for GitHub project title.
Git CLI Commands and Version Control Workflow	Good for command-focused practical showcase.
Basic to Advanced Git Command Practice Project	Good for learning portfolio.
Git Branching, Merging, Collaboration and History Management Project	Good for advanced project summary.

12. LinkedIn-ready Project Summary

You can use the following summary while posting this project on LinkedIn.

Project: Git Version Control & Repository Management Project

I completed a hands-on Git practical project covering repository initialization, staging, commits, history inspection, branching, merging, stashing, collaboration with remote repositories, and safe history management using reset, revert, reflog, amend, and interactive rebase.

This project helped me understand how professional developers and DevOps teams manage source code, track changes, collaborate using branches, recover from mistakes, and maintain clean project history.

Key Skills Practiced:
- Git CLI and repository management
- Staging, commits, logs, diff, restore, and clean
- Branching, merging, stash, rebase, and cherry-pick
- Remote collaboration using GitHub
- Tags, reflog, revert, reset, amend, and interactive rebase

Tools Used: Git, Git Bash / Terminal, GitHub, VS Code

Suggested hashtags: #Git #GitHub #VersionControl #DevOps #CloudEngineer #Linux #SoftwareDevelopment #OpenSource #DeveloperJourney